Name: Naveen Kumar P **Reg. No.:** 192412326

Experiment No.: 2

Experiment Question

For a given set of training data examples stored in a .CSV file, implement and demonstrate the Candidate-Elimination algorithm in Python to output a description of the set of all hypotheses consistent with the training examples.

Aim

To implement the Candidate-Elimination algorithm in Python and use it to find the **version space**, that is, the set of all hypotheses consistent with the training examples, represented by its specific boundary S and its general boundary G.

Concept

The **version space** VS is the subset of the hypothesis space H that is consistent with all the training examples. Instead of listing every hypothesis, the Candidate-Elimination algorithm stores only two boundaries:

- **S (specific boundary):** the set of the maximally specific hypotheses consistent with the data.
- **G (general boundary):** the set of the maximally general hypotheses consistent with the data.

Every hypothesis that lies between S and G (more general than or equal to some member of S, and more specific than or equal to some member of G) belongs to the version space.

The algorithm starts with S as the most specific hypothesis and G as the most general hypothesis. A **positive** example generalizes S and removes from G any hypothesis that does not cover the example. A **negative** example specializes G and removes from S any hypothesis that covers the example. Unlike FIND-S, this algorithm uses both positive and negative examples.

Dataset Description

The EnjoySport dataset is stored in a CSV file named `enjoysport.csv`. It has 4 training examples, 6 input attributes and 1 target attribute (EnjoySport = Yes/No).

Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
Sunny	Warm	Normal	Strong	Warm	Same	Yes
Sunny	Warm	High	Strong	Warm	Same	Yes
Rainy	Cold	High	Strong	Warm	Change	No
Sunny	Warm	High	Strong	Cool	Change	Yes

Contents of enjoysport.csv:

```
Sky,AirTemp,Humidity,Wind,Water,Forecast,EnjoySport
Sunny,Warm,Normal,Strong,Warm,Same,Yes
Sunny,Warm,High,Strong,Warm,Same,Yes
Rainy,Cold,High,Strong,Warm,Change,No
Sunny,Warm,High,Strong,Cool,Change,Yes
```

Algorithm

```
Candidate-Elimination Algorithm
1. Initialize G to the maximally general hypothesis:  G = { <?, ?, ?, ?, ?, ?> }
   Initialize S to the maximally specific hypothesis: S = { <0, 0, 0, 0, 0, 0> }
2. For each training example d:

   If d is a POSITIVE example:
     a. Remove from G every hypothesis that is inconsistent with d
     b. For each attribute in S that is not satisfied by d:
          replace it by the next more general constraint that d satisfies

   If d is a NEGATIVE example:
     a. Remove from S every hypothesis that is inconsistent with d
     b. For each hypothesis g in G that covers d:
          remove g from G and add to G all minimal specializations h of g such that
            - h does not cover d, and
            - h is more general than the S hypothesis
3. Output the boundary sets S and G
```

Step-by-step Procedure

1. Read the training examples from enjoysport.csv.
2. Initialize S = ['0','0','0','0','0','0'] and G = [['?','?','?','?','?','?']].
3. Take the training examples one by one and check the target value.
4. For a **positive** example:
    - Remove from G the hypotheses that do not cover the example.
    - Generalize S: replace '0' with the attribute value, or replace a mismatching value with '?'.
5. For a **negative** example:
    - Keep S unchanged, since S does not cover the negative example.
    - For each g in G that covers the example, replace g with its minimal specializations. In each one, a '?' is replaced by the S value of that attribute, only where S is not '?' and differs from the example.
6. Print S and G after every example.
7. After all examples are processed, print the final S and G. They describe the version space.

Formulae

A hypothesis h is consistent with a training set D when h(x) = c(x) for every example ⟨x, c(x)⟩ in D.

Version space: VS(H, D) = { h ∈ H | Consistent(h, D) }

Specific boundary: S = { s ∈ H | Consistent(s, D) ∧ ¬∃ s' ∈ H [(s > s') ∧ Consistent(s', D)] }

General boundary: G = { g ∈ H | Consistent(g, D) ∧ ¬∃ g' ∈ H [(g' > g) ∧ Consistent(g', D)] }

A hypothesis h covers an example x if, for every attribute i, h(i) = ? or h(i) = x(i).

Python Program

```
import csv

# Step 1: Read the training data from the CSV file
with open('enjoysport.csv') as f:
    reader = csv.reader(f)
    header = next(reader)
    data = [row for row in reader]

n = len(header) - 1          # number of attributes

def covers(h, x):
    """Return True if hypothesis h covers example x."""
    return all(h[i] == '?' or h[i] == x[i] for i in range(n))

# Step 2: Initialize S and G
S = ['0'] * n
G = [['?'] * n]

print("Initial S =", S)
print("Initial G =", G)

# Step 3: Candidate-Elimination algorithm
for t, row in enumerate(data, 1):
    x, label = row[:-1], row[-1]

    if label == 'Yes':                    # positive example
        G = [g for g in G if covers(g, x)]   # remove inconsistent members of G
        for i in range(n):                # generalize S
            if S[i] == '0':
                S[i] = x[i]
            elif S[i] != x[i]:
                S[i] = '?'
    else:                                 # negative example
        new_G = []
        for g in G:
            if not covers(g, x):
```

```
            new_G.append(g)              # g already excludes x
        else:
            for i in range(n):           # minimal specializations of g
                if g[i] == '?' and S[i] != '?' and S[i] != x[i]:
                    h = g.copy()
                    h[i] = S[i]
                    new_G.append(h)
    G = new_G

  print(f"\nInstance {t} ({label}):")
  print("  S =", S)
  print("  G =", G)

# Step 4: Final version space boundaries
print("\nFinal Specific Boundary S:", S)
print("Final General Boundary G :", G)
```

Output

```
Initial S = ['0', '0', '0', '0', '0', '0']
Initial G = [['?', '?', '?', '?', '?', '?']]

Instance 1 (Yes):
  S = ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same']
  G = [['?', '?', '?', '?', '?', '?']]

Instance 2 (Yes):
  S = ['Sunny', 'Warm', '?', 'Strong', 'Warm', 'Same']
  G = [['?', '?', '?', '?', '?', '?']]

Instance 3 (No):
  S = ['Sunny', 'Warm', '?', 'Strong', 'Warm', 'Same']
  G = [['Sunny', '?', '?', '?', '?', '?'], ['?', 'Warm', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', 'Same']]

Instance 4 (Yes):
  S = ['Sunny', 'Warm', '?', 'Strong', '?', '?']
  G = [['Sunny', '?', '?', '?', '?', '?'], ['?', 'Warm', '?', '?', '?', '?']]

Final Specific Boundary S: ['Sunny', 'Warm', '?', 'Strong', '?', '?']
Final General Boundary G : [['Sunny', '?', '?', '?', '?', '?'], ['?', 'Warm', '?', '?', '?', '?']]
```

Result / Conclusion

The Candidate-Elimination algorithm was implemented in Python and run on the EnjoySport dataset. The version space is bounded by:

- **S = { ⟨Sunny, Warm, ?, Strong, ?, ?⟩ }**
- **G = { ⟨Sunny, ?, ?, ?, ?, ?⟩, ⟨?, Warm, ?, ?, ?, ?⟩ }**

Six hypotheses are consistent with the training examples: the one in S, the two in G, and three lying between them: ⟨Sunny, Warm, ?, ?, ?, ?⟩, ⟨Sunny, ?, ?, Strong, ?, ?⟩ and ⟨?, Warm, ?, Strong, ?, ?⟩. Hence the Candidate-Elimination algorithm was successfully demonstrated.